

# Overview

As part of this course, you will work in teams to implement a web application. As this is a class project, the web application is not expected to be production-ready, released to, or evaluated with real users. Instead, the objective is to practice software engineering concepts and apply the knowledge you've gained throughout the course.

You will learn important skills such as teamwork, communication, project management, and coding standards. Each team will be responsible for the full development lifecycle, including requirements gathering, design, implementation, testing, and documentation.

The high-level description for each project is provided. You are expected to expand on this description and define the specific features, functionality, and user interfaces of your application. Each team should create detailed user stories that outline different user roles and the tasks each role will perform within the application. For each requirement, you would also need to design comprehensive test cases that include not only the default behavior but also edge cases. You should also create a technical architecture/design document that outlines the structure and components of your application. This document should also include diagrams, such as UML diagrams, database design, technical constraints, and assumptions.

At a minimum, the technology stack should include:

- React and Typescript for the front end
- Python (FastAPI, SQLAlchemy, and Pydantic) for the service/backend
- PostgreSQL for the database

You are encouraged to use open-source libraries and frameworks to enhance functionality and streamline development, such as UI component libraries, authentication and user management libraries, and unit testing frameworks.

You are expected to use an Integrated Development Environment (IDE), such as Visual Studio Code, for development. You can also utilize your hawaii.edu email to obtain a license for the JetBrains IDEs and development tools for free.

Please note that this is not a UI/UX/Design class, so you should not spend excessive effort on implementing a sophisticated user interface. The richness/sophistication of the UI will not be evaluated; instead, focus on the functionality and underlying code structure. At a minimum, you should ensure that the web application is responsive (i.e., automatically adapts its layout and functionality to fit any device).

It is expected that each team member has the development environment set up and functioning on their workstation/laptop.

You must utilize GitHub as your version control system. One team member can create the repository and invite the rest of the team to collaborate. It is essential that you leverage GitHub's features effectively, including branching for managing different features or fixes, pull requests for code reviews and collaborative development, issues for tracking tasks and bugs, Projects for

project management, milestones to track progress toward key deadlines, and wiki for project documentation. You are also encouraged to use GitHub Actions for continuous integration and deployment (CI/CD).

Your instructor will create a channel on the class Discord server for your project. You are expected to use this channel for all project-related communication among the team. You should also create a pinned message that includes the URL of the GitHub repository.

## Project: Neighborhood Tool Sharing

For this semester-long group project, your team will design, develop, and deploy a responsive web application that supports invite-only tool lending within a private community. Users can create an account only with a valid invite, set up a basic profile, and list tools (e.g., drills, ladders, garden equipment) with photos, description, condition, and optional lending rules (loan duration limits, pickup/return notes).

Members can browse and search available tools and request a reservation for a specific time window. Each reservation follows the sequence REQUESTED → APPROVED → PICKED\_UP → RETURNED, with REQUESTED → DENIED as the terminal rejection path, and CANCELLED allowed only from REQUESTED or APPROVED (cancellation after pickup is not allowed). The system must prevent double-booking by ensuring no overlapping APPROVED/PICKED\_UP reservations exist for the same tool. Each reservation includes a private message thread for coordination and generates notifications for key status changes (request submitted, approved/denied, pickup/return reminders if implemented). After a reservation is returned, both borrower and owner can leave a short rating/review.

The application must include a member dashboard showing their listed tools, incoming requests, and outgoing reservations by status. An admin role can suspend users, deactivate listings (hide from browsing without deleting audit history), and generate basic reports.

The deliverable is a deployed web app with a clear README, seeded demo data, and automated tests covering booking rules, permissions, and the main end-to-end flow. Your team is expected to apply software engineering practices from the course (requirements, design, version control, code reviews, testing, and iterative delivery) through incremental milestones aligned with the course schedule.

*Remember, the goal is not to build a commercial-grade product, but to gain hands-on experience with the full software development lifecycle while working collaboratively as a team.*

### Sample User Stories

#### User Story #1:

As a tool owner, I want to approve or deny reservation requests so that I can control when my tools are borrowed.

#### Acceptance Criteria:

##### **Scenario 1: Owner approves request**

Given a reservation is in REQUESTED state

And the logged-in user is the owner of the tool  
When the owner approves the reservation  
Then the reservation status becomes APPROVED

**Scenario 2: Owner denies request**

Given a reservation is in REQUESTED state  
And the logged-in user is the owner of the tool  
When the owner denies the reservation  
Then the reservation status becomes DENIED

**Scenario 3: Non-owner cannot approve request**

Given a reservation is in REQUESTED state  
And the logged-in user is not the owner of the tool  
When the user attempts to approve the reservation  
Then the system rejects the action  
And the reservation status remains REQUESTED

User Story #2:

As a borrower, I want to request a reservation for a tool for a specific date range so that I can borrow the tool when I need it.

Acceptance Criteria:

**Scenario 1: Valid reservation request**

Given the tool has no approved or picked-up reservations for the requested dates  
When the borrower submits a reservation request from May 1 to May 3  
Then the reservation is created with status REQUESTED

**Scenario 2: Invalid date range**

Given the borrower is on the reservation request page  
When the borrower enters an end date before the start date  
Then the system rejects the request and displays a validation error

Sample Test Cases

| Field         | Example             |
|---------------|---------------------|
| Test case ID  | TC-001              |
| Related story | Create tool listing |

|                 |                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------|
| Precondition    | User is logged in as a member                                                                      |
| Test data       | Color: Brown; Name: Drill, Brand: DeWalt, etc.                                                     |
| Steps           | 1. Open create listing page. 2. Enter tool name, description, condition, category, etc. 3. Submit. |
| Expected result | Tool listing is created and visible in the owner's listed tools.                                   |
| Actual result   | <i>Filled in during testing</i>                                                                    |
| Pass/fail       | <i>Filled in during testing</i>                                                                    |

| Field           | Example                                   |
|-----------------|-------------------------------------------|
| Test case ID    | TC-002                                    |
| Related story   | Browse available tools                    |
| Precondition    | Active tool listings exist                |
| Test data       | -                                         |
| Steps           | 1. Log in as member. 2. Open browse page. |
| Expected result | Active listings are displayed.            |
| Actual result   | <i>Filled in during testing</i>           |
| Pass/fail       | <i>Filled in during testing</i>           |

## Project: Local Produce Exchange

For this semester-long group project, your team will design, develop, and deploy a responsive web application that reduces food waste by enabling an invite-only exchange of surplus produce and other food items within a trusted private community.

Users join only via invitation, create a profile, and can post listings for items they want to share with required details: description, category (e.g., fruits, vegetables, baked goods, dairy), photos (optional), quantity available, dietary/allergen tags (e.g., vegan, contains nuts, gluten-free), and an available pickup window. Members can browse/search listings and submit a claim request for a specified quantity. Each request follows the sequence REQUESTED → APPROVED → PICKED\_UP → COMPLETED, with REQUESTED → DENIED as a terminal rejection path, and CANCELLED allowed only from REQUESTED or APPROVED (cancellation after pickup is not allowed). The system must prevent conflicting claims by ensuring that approved quantities never exceed the remaining quantity. Each exchange includes a private message thread for coordination and generates notifications for status changes and pickup reminders (if implemented). After completion, both sides can leave a short rating/review.

The application must include user profiles showing a member's active listings, incoming/outgoing requests, and exchange history, plus a dashboard to manage actions by status (approve/deny, cancel, picked up, completed). An admin role can suspend users, deactivate listings (hide from browsing without deleting audit history), and generate basic reports.

The deliverable is a deployed web app with a clear README, seeded demo data, and automated tests covering permissions and the main end-to-end flow. Your team is expected to apply software engineering practices from the course (requirements, design, version control, code reviews, testing, and iterative delivery) through incremental milestones aligned with the course schedule.

*Remember, the goal is not to build a commercial-grade product, but to gain hands-on experience with the full software development lifecycle while working collaboratively as a team.*

### Sample User Stories

User Story #1:

As a claimant, I want to submit a claim for a specified quantity of a listing so that I can request only the amount I need.

Acceptance Criteria:

#### **Scenario 1: Valid claim request**

Given a listing is active

And the listing has 10 units available  
When the claimant requests 3 units  
Then a claim is created with status REQUESTED  
And the requested quantity is 3

**Scenario 2: Claim quantity exceeds available quantity**

Given a listing is active  
And the listing has 10 units available  
When the claimant requests 12 units  
Then the system rejects the claim  
And no claim is created

**Scenario 3: Claim quantity must be positive**

Given a listing is active  
When the claimant requests 0 units  
Then the system rejects the claim  
And displays a validation error

User Story #2:

As a claimant or poster after completion, I want to leave a short rating and review of the other party, so that the community can build trust.

Acceptance Criteria:

**Scenario 1: Claimant leaves a review after completion**

Given an exchange has status COMPLETED  
And the logged-in user is the claimant for that exchange  
And the claimant has not already reviewed the poster  
When the claimant submits a rating and short review for the poster  
Then the review is saved  
And the review is associated with that completed exchange  
And the poster can view the review

**Scenario 2: Poster leaves a review after completion**

Given an exchange has status COMPLETED  
And the logged-in user is the poster for that exchange  
And the poster has not already reviewed the claimant  
When the poster submits a rating and short review for the claimant  
Then the review is saved  
And the review is associated with that completed exchange  
And the claimant can view the review

**Scenario 3: Review is not allowed before completion**

Given an exchange has status REQUESTED, APPROVED, PICKED\_UP, DENIED, or CANCELLED

When either party attempts to submit a review

Then the system rejects the review

And no review is saved

**Scenario 4: Non-participant cannot review**

Given an exchange has status COMPLETED

And the logged-in user is not the poster or claimant for that exchange

When the user attempts to submit a review

Then the system denies access

And no review is saved

**Sample Test Cases**

| Field           | Example                                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| Test case ID    | TC-001                                                                                                          |
| Related story   | Leave rating and review after completion                                                                        |
| Precondition    | Exchange #42 exists with status COMPLETED; user A is the claimant; user B is the poster                         |
| Test data       | Rating: 5; Review text: "Easy pickup and fresh produce."                                                        |
| Steps           | 1. Log in as user A. 2. Open exchange #42. 3. Click "Leave Review." 4. Enter rating and review text. 5. Submit. |
| Expected result | Review is saved, linked to exchange #42, and visible to the poster                                              |
| Actual result   | <i>Filled in during testing</i>                                                                                 |
| Pass/fail       | <i>Filled in during testing</i>                                                                                 |

| Field        | Example |
|--------------|---------|
| Test case ID | TC-002  |



|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| Related story   | Invite-only registration                                                            |
| Precondition    | Valid unused invite token exists                                                    |
| Test data       | First Name: John; Last Name: Smith, etc.                                            |
| Steps           | 1. Open registration page. 2. Enter account information and valid token. 3. Submit. |
| Expected result | Account is created. Token is marked as used.                                        |
| Actual result   | <i>Filled in during testing</i>                                                     |
| Pass/fail       | <i>Filled in during testing</i>                                                     |

# Project Deliverables

## 1) Team + Process Setup

### A. Team charter

### B. Repository setup

- Repository, issue, project board, etc., setup
  - README stub
  - Configure the repository to prevent commits directly to main
- 

## 2) Requirements

### A. Core Requirements

- High-level use cases
- 25 to 30 user stories (including acceptance criteria)
  - Edge cases and negative/error-path flows
- Domain model

### B. Cross-Team Requirements Review Packet

### C. Cross-Team Requirements Review Report

---

## 3) Technical Design

### A. Lightweight design doc

- Architecture diagram + components
- Technology stack (including libraries and frameworks)
- Data model (ERD)
- Key API endpoints
- Risks/tradeoffs listed

### B. Code quality tools

- Tools that will be utilized
- When and where will they be executed

---

## 4) Implementation

- Use of GitHub as the code repository
  - Branch-based development and code reviews before merging to main
  - Automated unit tests for business rules
  - Use of appropriate commit messages
  - Use of code quality tools
- 

## 5) Testing

### A. Test cases for manual testing

- Create the key test cases
  - Nice to have is using a tool like Playwright to automate UI testing
- 

## 6) Deployment

### A. Deployment guide

- Create (and validate) a step-by-step guide for deploying the web application
  - Document detailing the implemented features, known limitations, and risks
- 

## 7) Communication Deliverables

### A. In-class presentations and demos

- Clear walkthrough covering required scenarios
- Ability to answer questions posed by the instructor (and attendees)

### B. Peer evaluations

- Evaluations for each team member
-

# Grade Distribution

| Project component                                          | Due / milestone | Weight    | Main grading focus                                                                                                                   |
|------------------------------------------------------------|-----------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Team + process setup</b>                             | 6/1 and 6/4     | <b>3%</b> | Team charter, GitHub repo, project board, README stub, branch protection, Discord/repo link                                          |
| <b>2. Inception presentation + requirements packet</b>     | 6/8             | <b>8%</b> | Project scope, roles/personas, core use cases, 25–30 user stories, acceptance criteria, assumptions, risks                           |
| <b>3. Cross-team requirements review</b>                   | 6/15            | <b>4%</b> | Quality of feedback given to another team: ambiguity, missing cases, workflow gaps, scope risks                                      |
| <b>4. Technical design document</b>                        | 7/1             | <b>7%</b> | Architecture diagram, components, ERD/data model, API endpoints, tech stack, code quality tools, tradeoffs                           |
| <b>5. R1 demo + presentation + code walkthrough</b>        | 7/6–7/9         | <b>7%</b> | First working vertical slice, evidence of implementation progress, ability to explain code, retrospective/defect triage              |
| <b>6. Manual test cases / QA packet</b>                    | 7/14            | <b>4%</b> | Test cases tied to user stories, positive and negative paths, edge cases, permission/workflow tests                                  |
| <b>7. R2 demo + presentation + code walkthrough</b>        | 7/27            | <b>7%</b> | Broader feature completion, improved quality from R1, defect resolution, updated risks/scope                                         |
| <b>8. Deployment guide + release readiness</b>             | 8/6             | <b>3%</b> | Clear deployment steps, seeded demo data, known limitations, validated setup instructions                                            |
| <b>9. Final submission + final presentation</b>            | 8/11–8/13       | <b>9%</b> | End-to-end demo, completed requirements, final codebase, README, limitations, QA summary, lessons learned                            |
| <b>10. Individual contribution and engagement evidence</b> |                 | <b>8%</b> | GitHub activity, issue ownership, code reviews, testing/documentation work, presentation accountability, peer evaluation, reflection |

| <b>Individual contribution category</b>                                                                           | <b>Weight</b> |
|-------------------------------------------------------------------------------------------------------------------|---------------|
| GitHub contribution evidence: meaningful PRs, commits, reviewed PRs, tests, documentation                         | <b>3%</b>     |
| Issue ownership and project management: assigned issues, project board updates, meeting notes, milestone tracking | <b>1.5%</b>   |
| Code review / testing / QA contribution                                                                           | <b>1.5%</b>   |
| Presentation, demo participation, and ability to answer questions                                                 | <b>1%</b>     |
| Peer evaluation + individual reflection                                                                           | <b>1%</b>     |

# Project Presentations

***Note: All team members should collaborate in preparing the presentation. Each team member should present a similar number of slides.***

## Project Initiation

*At a minimum, the presentation should contain the following.*

- Project objectives
  - Outline the goals of the project
  - What value will this project bring to the client/end users?
- Project scope
  - Clearly define what is included in the project scope and what is not
- User roles/personas
  - Identify the different user roles and personas involved
- Key use cases and user stories
- Acceptance criteria
- Project management overview
  - Team member roles and responsibilities
  - Recurring meeting schedule and plan for tracking meeting minutes
  - Task management and tracking
- Key risks and mitigation strategies
- Project Artifacts
  - List the documents and tools the team will create (e.g., requirements specification, design specification, test cases, deployment plan, etc.)
- Quality Assurance Plan
  - Outline the approach for quality assurance, defect tracking, and code quality
- Milestones
  - Define what will be delivered at each milestone
- Next steps

## Mid-Semester - Presentation + Live Demo

*At a minimum, the presentation should contain the following.*

- Overview of the domain model and the ER diagram
- Major accomplishments
  - Include screenshots
  - Include brief details about the technical implementation
- Major changes, if any, to the requirements
- Changes, if any, to the project management approach since the initiation
- Updated key risks and mitigation strategies, if any

- Were there any risks identified in Project Initiation that materialized? How were they addressed?
- Quality assurance metrics
  - Defects reported, fixed, open, etc.
  - Code quality metrics, unit test coverage
- Milestones
  - Missed/Pospouted milestones
    - Why?
  - Upcoming milestones
    - What will be delivered at these milestones?
- Next Steps

In addition to demonstrating key functionality in the live demo, teams should be prepared to conduct a code walkthrough and explain key components of the codebase. The instructor and audience will ask questions regarding functionality and the codebase.

There will be two mid-semester demos - R1 and R2

## Final Presentation - Presentation + Live Demo

*At a minimum, the presentation should contain the following.*

- Project objectives
  - Provide a brief overview of the project's goals
  - What original project goals/features were not achieved and why?
- Technical solution overview
- Major Accomplishments
  - Highlight key achievements, including any requirements that were not implemented or were partially done, along with the rationale for these decisions.
  - Include key screenshots
- Quality Assurance Metrics (Overall)
  - Provide an overall summary of defects reported, fixed, and currently open
- Project Management and Delivery
  - Overview of the software process the team followed, including the use of AI in development.
- Reflections
  - What went right?
  - What went wrong?
  - What could have been done differently?
  - Discuss both technical and soft skills acquired or improved by team members
- Acknowledgments (if any)

In addition to demonstrating key functionality in the live demo, teams should be prepared to conduct a code walkthrough and explain key components of the codebase. The instructor and audience will ask questions regarding functionality and the codebase.

# Cross-Team Requirements Peer Review

Each team will review the other team's project requirements and provide structured feedback. The goal is to identify ambiguity, missing requirements, unrealistic scope, inconsistent workflows, unclear user roles, and missing edge cases before implementation begins.

The reviewing team will act as informed stakeholders and peer software engineers. Their responsibility is not to design the solution but to offer a fresh perspective before the other team commits to design and implementation decisions.

## Artifacts for review

Each team should submit a Requirements Review Packet containing:

1. User stories
2. Acceptance criteria
3. User roles/personas
4. Open questions and assumptions
5. Known risks

## Conducting the review

The following are suggestions to assist the reviewing team:

1. User stories - do they describe behavior clearly enough to facilitate testing?
2. Acceptance criteria - are they specific and complete enough to determine when the story is done?
3. User roles/personas - have all relevant user roles (member, owner/poster, admin, guest) been accounted for?
4. Edge cases - what scenarios have not been planned for?
5. Requirements consistency - are there contradictions between stories? Does any story depend on elements not included in the backlog?

## Review feedback

The Requirements Review Report, typically 2–4 pages, should include the following:

1. Summary - an overall impression of the requirements
2. Strengths - Clear, useful, or well-scoped elements
3. Ambiguities - requirements that could be interpreted in multiple ways
4. Missing cases - edge cases, failure cases, permissions, or workflow gaps
5. Scope risks - features that seem too large or technically risky



# Presentation and class discussion

Each team will present its Requirements Review Report in a 10-minute presentation, followed by a brief Q&A session. During this discussion, the team whose requirements are being reviewed should:

- Be prepared to clarify any aspects of their requirements that may have been questioned during the review.
- Take notes on feedback and suggestions for subsequent revisions of their requirements.

# Team Charter

## **Template:**

[https://docs.google.com/document/d/1O\\_wL98pSWUaWf7yBahiy3-LGO1QjNDpl/edit?usp=sharing&ouid=103020945264628236703&rtpof=true&sd=true](https://docs.google.com/document/d/1O_wL98pSWUaWf7yBahiy3-LGO1QjNDpl/edit?usp=sharing&ouid=103020945264628236703&rtpof=true&sd=true)

## **Sample:**

<https://docs.google.com/document/d/1lLudKSwsn1fTSAduMEkGIOIS7VJFAUCk/edit?usp=sharing&ouid=103020945264628236703&rtpof=true&sd=true>